

端侧高性能优化方法-基于Roofline模型

i 出发点：找到模型“大小”和推理速度间的联系

1. 模型“大小”的四种描述指标

1.1 计算量

- 模型运行一次所需的计算次数
- 反映了模型对硬件计算单元能力的需求
- 单位：OPs (Operations) 或 FLOPs (Floating Point Operations)
- 计算方法：模型中所有算子的计算量之和
 - 单个算子的计算量：每种算子各不一致。以卷积为例，公式如下：
$$FLOPs_of_Conv = N \times OC \times OH \times OW \times IC / Groups \times KH \times KW \times 2$$
- 计算量不单独影响模型推理速度，计算量低 $\neq$ 推理速度快

1.2 参数量

- 模型中所有参数的总和
 - 在 CNN 模型中，主要是 Conv/FC/DeConv 的 Weight，其他层占比很小
- 是模型文件大小的决定性因素
 - 参数量大的模型会：
 - 占用更多的磁盘空间
 - 使得 APK 包体积膨胀
- 缓解路径：
 - 设计网络时减少参数量
 - 模型压缩
- 不直接影响推理速度，但会影响内存占用和模型初始化时间

1.3. 访存量

- 模型运行一次需要访问存储单元的字节数
- 反映了模型对存储单元带宽的需求
- 单位：Bytes (或者 KB/MB/GB)
- 单个算子的访存量：输入 Tensor + 输出 Tensor 的字节数。以卷积为例，公式如下：
$$MACs_of_CONV = MACs_of_Input + MACs_of_Output + MACs_of_Weight = (N \times IC \times IH \times IW + N \times OC \times OH \times OW + OC$$

- 对推理速度至关重要，且易被忽略

1.4. 内存占用

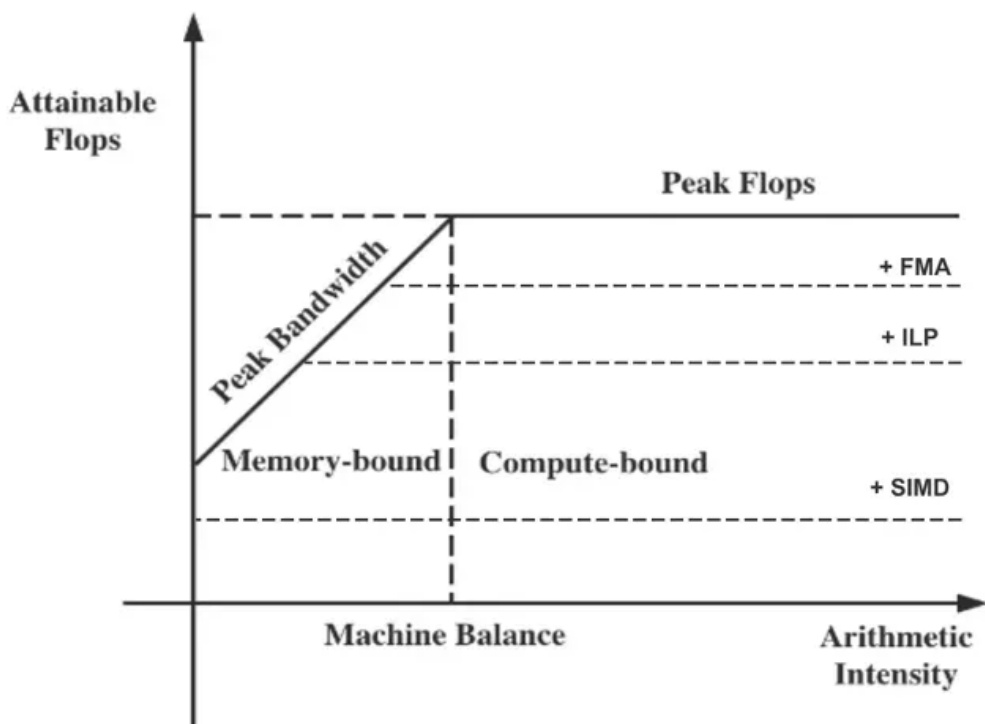
- 模型运行时所占用的内存/显存大小
- 平均占用 & 最大占用（重点考虑）
- 内存占用受模型本身以及软件实现影响
- 对推理速度没有直接影响，但在很多场景下有要求

2. 计算量、访存量与推理速度的关系

- ❗ 如何衡量给定计算任务在目标设备上的性能天花板？即算子在目标设备上的性能上线是多少？算子实现在目标设备上是否还有优化空间？

2.1 Roofline模型背景介绍

- ❗ 模型简单，高度抽象（与ISA无关，与软件实现无关）



Roofline模型

- 纵轴(计算速度)：可达到的性能上限(Attainable FLOP/s, max FLOPs per Second)，取与硬件理论峰值相比的最小值。

- ❗
$$FLOP/sec = \frac{FLOP}{sec} = \frac{FLOP}{Byte} \times \frac{Byte}{sec}$$
$$FLOP/sec = AI \times BW$$

- 横轴(计算密度): 计算量/访存量, 描述在硬件平台上, 单位内存交换最多用来进行多少次计算。

❗ 以Elementwise Add为例, 计算密度I的计算公式如下:

$$I = \frac{FLOPs}{MACs} = \frac{N \times C \times H \times W}{N \times C \times H \times W \times 3 \times \text{sizeof(float)}} = \frac{1}{12} FLOP/Byte$$

- 斜率(内存带宽): 目标存储层次带宽(Byte/s, Max memory Access per Second)。
- 将硬件的计算/访存能力与计算任务的需求进行可视化与分析。

常用术语

- 理论算力:

核数(*cores*) * 主频(*GHz*) * 2(*fmlaport*) * 2(*MulAndAdd*) * 128(*bits/fmla512*)/32(*bits_of_Flo*

- 内存带宽:

内存时钟频率 × 内存总线位数 × 倍增系数/8

- 计算密度:

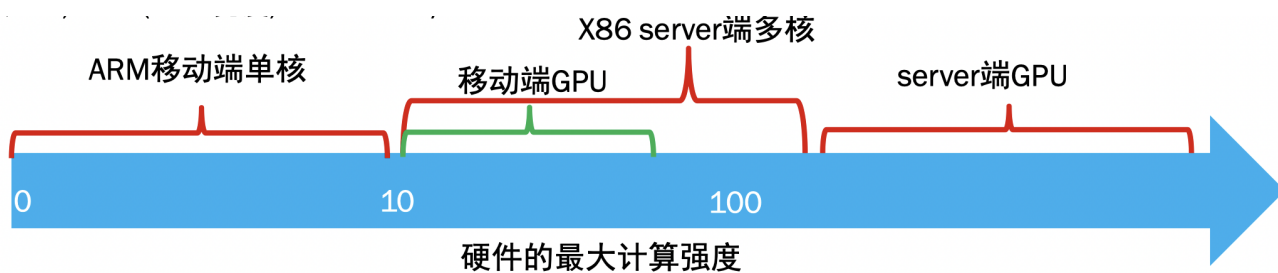
计算密度 $I(FLOP/Byte) = \text{计算量}(FLOP) \div \text{访存量}(Byte)$

❗ 以iQOO Neo 3 (高通865) 为例:

Cortex-A77 fp32单核理论算力: 1core x 2.84GHz x 2(2个FPU port) x 2(每个port在每个cycle 计算一次fmla) x 4 (SIMD宽度) = 45.44 GFLOP/s

最大内存带宽(冷启动, 假设数据都在DRAM): 25.96 GB/s

A77计算密度上限: 45.44/25.96=1.75 GFLOP/Byte



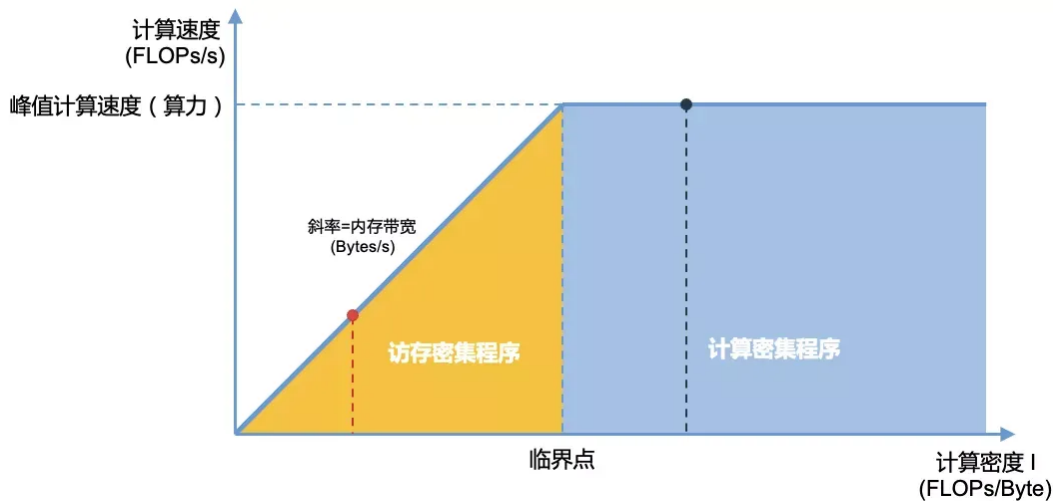
- 计算速度:

计算速度($FLOP/s$) = $\min(\text{峰值计算速度}, \text{计算密度} * \text{内存带宽})$

- 计算时间:

计算时间 = $\frac{\text{计算量}}{\text{计算速度}} = \frac{\text{计算量}}{\min(\frac{\text{计算量}}{\text{访存量}} \times \text{内存带宽}, \text{理论峰值计算速度})}$

2.2 计算密集型与访存密集型算子



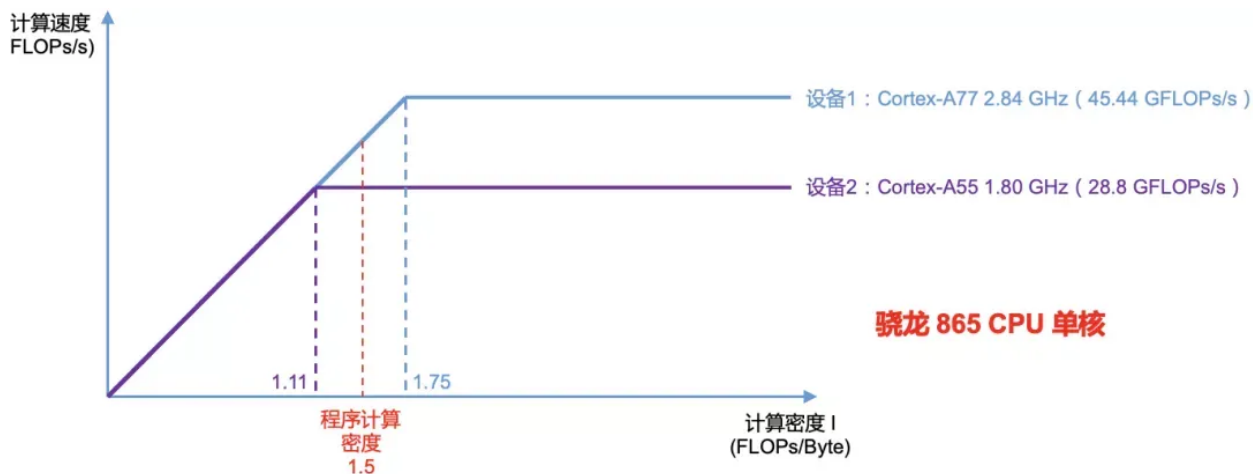
$$\text{算子理论计算时间} = \begin{cases} \frac{\text{访存量}}{\text{内存带宽}} & \text{访存密集型算子} \\ \frac{\text{计算量}}{\text{理论峰值计算速度}} & \text{计算密集型算子} \end{cases}$$

- 根据 RoofLine 模型，计算量、访存量与算子理论计算时间的关系：
 - 访存密集型算子：计算时间跟访存量呈线性关系
 - 计算密集型算子：计算时间跟计算量呈线性关系
- 网络中的算子可根据计算密度，分为计算密集型算子和访存密集型算子
 - 典型的算子分类
 - 访存密集型算子：Concat、Eltwise Add、ReLU、MaxPooling.....
 - 计算密集型算子：Conv、DeConv、FC、MatMul、LSTM.....

2.3 基于Roofline分析算子性能

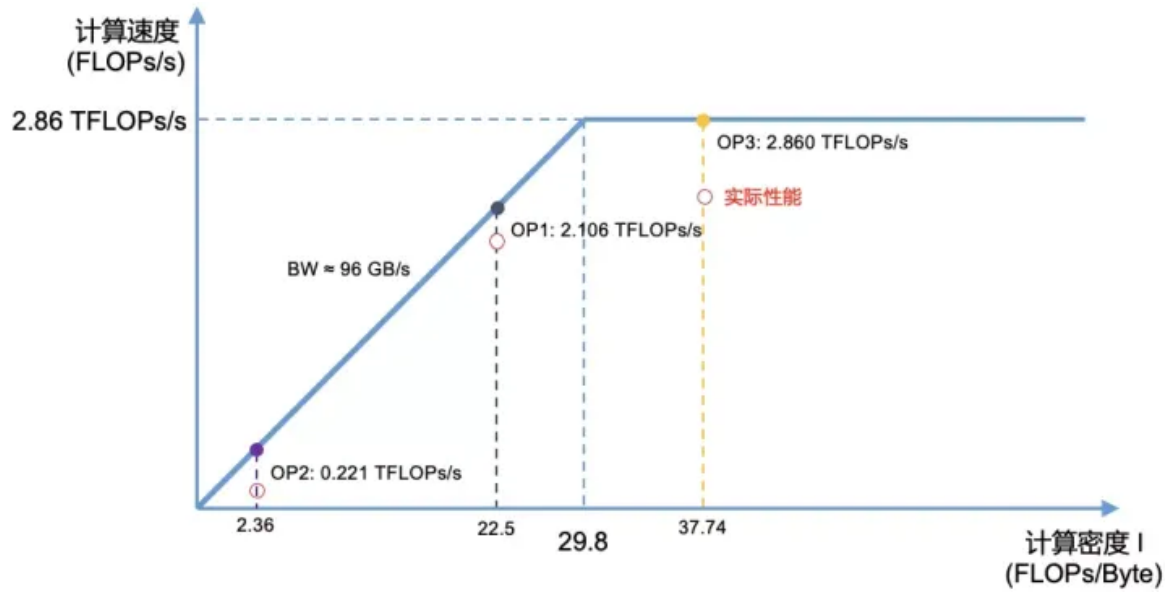
- 算子分类不绝对
 - 同一个算子在不同硬件上表现出的性质可能会不同
 - 同一个算子在不同参数下，计算密度不同，甚至会导致算子性质发生变化

i 在不同的硬件设备上，同一个算子的性质可能会不同



i 同一个算子在不同参数下，计算密度不同，甚至会导致算子性质发生变化

序号	batch	ic	ih,iw	kh,kw	stride	pad	dilation	group	oc	oh,ow	计算量(MFLOPs)	访存量(MB)	计算密度	理论计算速度	理论推理时间(us)
1	1	512	7	1	1	0	1	1	1024	7	51380	2287	22465	2106	2440
2	1	16	448	3	1	1	1	16	16	448	57803	24501	2359	0221	26134
3	1	16	448	3	1	1	1	1	16	448	924844	24509	37735	2860	32337



2.4 计算密度与推理时间

部署的同学们更喜欢谈“计算效率”，而实际上算法同学真正关心的点是“推理时间”，导致两者在对接的时候经常会出现一些 misleading。

按照 RoofLine 模型，我们能很容易得到算子的理论推理时间：

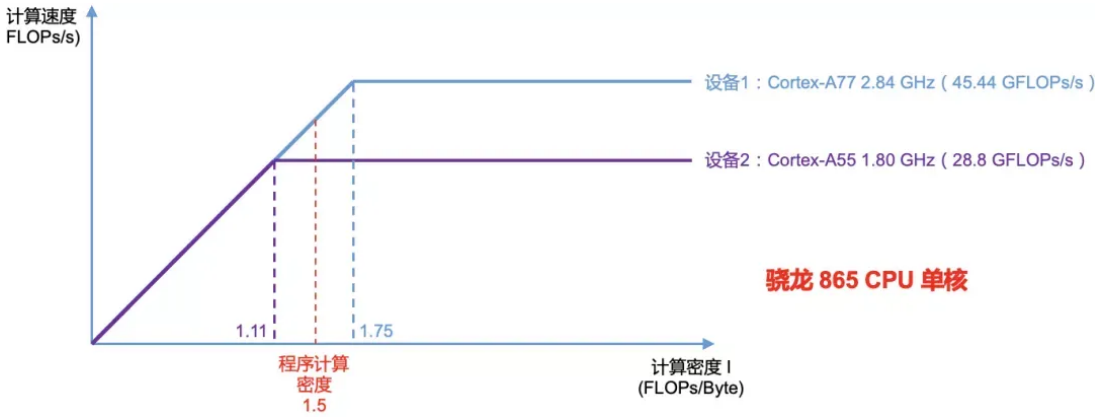
推理时间 = $\frac{\text{计算量}}{\text{计算速度}} = \frac{\text{计算量}}{\min(\frac{\text{计算量}}{\text{访存量}} \times \text{内存带宽}, \text{理论峰值计算速度})}$

因此，计算量、访存量与推理时间的关系为：

算子理论推理时间 = $\begin{cases} \frac{\text{访存量}}{\text{内存带宽}} & \text{访存密集型算子} \\ \frac{\text{计算量}}{\text{理论峰值计算速度}} & \text{计算密集型算子} \end{cases}$

- 访存密集型算子：推理时间跟访存量呈线性关系
- 计算密集型算子：推理时间跟计算量呈线性关系

同一种算子在不同参数下，计算密度不同，甚至会导致算子性质发生变化：
以上图的Cortex-A77 fp16浮点运算为例：



i 以iQOO Neo 3（高通865）为例：

Cortex-A77 fp16单核理论算力：1core x 2.84GHz x 2(2个FPU port) x 2(每个port在每个cycle 计算一次fmla) x 4 (SIMD宽度) = 45.44*2 GFLOP/s

最大内存带宽(冷启动，假设数据都在DRAM)： 25.96 GB/s

A77计算密度上限： 2*45.44/25.96=3.50 GFLOP/Byte

	op	-- bat ch	-- in_c han nel	{-- in_he ight -- in_wi dth}	{-- kern el_h -- kern el_w }	{-- stri de_ h -- stri de_ w}	{-- pad _h0 -- pad _h1 -- pad _w0 -- pad _w1 }	{-- dila _h - - dila _w}	-- gro up	-- out_c hann el	{oh, ow}	计算 量 (MFL OP)	访存 量 (MB)	计算 密度	实际 计算 时 (ms)
1															
2															
3															
4															
	1(con v)	1	512	7,7	1,1	1,1	0,0, 0,0	1,1	1	1024	7,7	51.38 0	1.14 4	44. 930	0.7 481 5
	2(de pth_c onv)	1	64	28,2 8	3,3	1,1	1,1, 1,1	1,1	64	64	28, 28	0.903	0.20 19	4.4 72	0.0 857
	4(con v)	1	32	7,7	1,1	1,1	0,0, 0,0	1,1	1	32	7,7	0.100 352	0.07 072 0	1.4 19	0.0 164 5

i 计算量并不能单独用来评估模型的推理时间，还必须结合硬件特性（算力、带宽以及访存量）来进行综合评估。

并非是计算量越低模型推理越快。

在评价模型大小时，建议加上访存量作为重要的评价指标。

3. 影响推理速度的其他因素

3.1 硬件因素

一些硬件因素会导致程序无法达到理论计算峰值：

- 功耗导致的频率变化

	AVX512	AVX	Normal
1~2	3700	3900	4500
3~4	3500	3700	4200
5~8	3400	3600	4100
9~12	3200	3600	4100
13~16	2900	3400	3900
17~18	2800	3300	3800

10980XE 不同核心数、不同指令集下的频率(MHz)

- 硬件在设计上计算与访存无法充分掩盖，例如 ARM 小核（Cortex-A53/A55）的静态执行流水线，以及低端 Risc-V 芯片的阻塞式 Cache

i 因此，硬件的理论峰值算力需要实测。

3.2 系统环境因素

1. Android 大小核调度问题

- a. CPU 占用率不足时，程序会被操作系统调度到小核
- b. 硬件在设计上计算与访存无法充分掩盖

i 解决方法：绑核

2. 缺页问题

- a. Linux 写时拷贝机制引发的缺页问题：第一次写入时，速度异常的慢

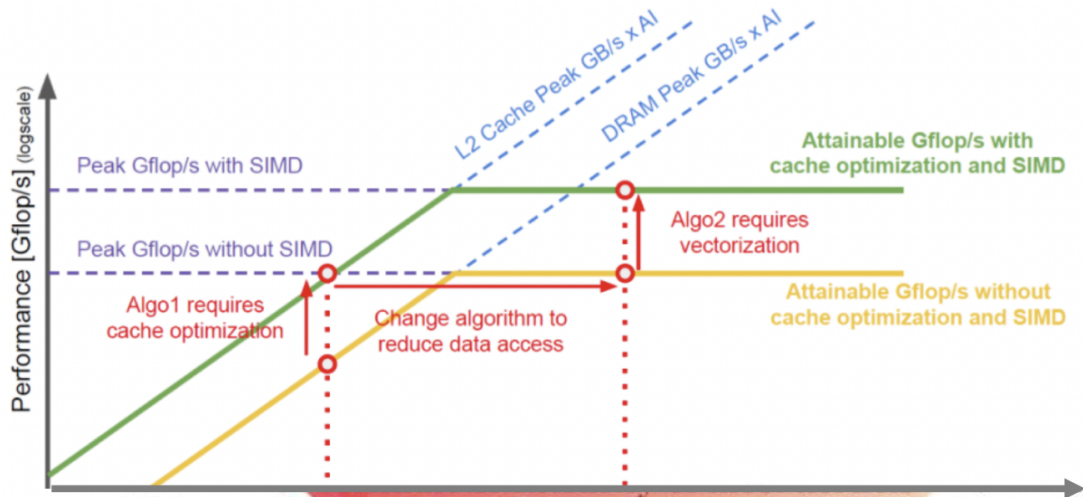
i 解决方法：大块内存不频繁申请 & 释放，或者使用内存池

3. 其他任务对硬件资源的挤占

- a. 对计算资源的挤占
- b. 对内存带宽等其他资源的挤占

i 解决办法：关注工程测试环境和实际部署系统环境的差异。如有条件，最好在实际部署环境下进行测试。

3.3 小结



RoofLine 模型仅能用于估计模型所能达到的性能上界，而实际部署时，还会受硬件限制、系统环境等因素的影响，导致无法达到 RoofLine 模型所定义的性能上界。

此外，由于这些因素往往会导致性能曲线有较强的非线性，理论分析和实测会有一定差距，有时这些因素会严重影响性能曲线，甚至会导致算子的性质发生变化。因此**实测始终是最准确的性能评估方式**。

- RoofLine 模型更重要的是提供了一种分析性能的思想，即计算密集型程序更多的受限于硬件算力，而访存密集型程序更多的受限于硬件内存带宽。在理解这一点的基础上设计网络结构，并分析网络的性能，将更有理论参考。不会再对“计算量减半，为啥推理时间没变”这种问题抱有疑问了。

4. 端到端实现高效的模型部署性能

4.1 端到端思想

- 不再单独以计算量作为模型“大小”的评价指标，而是多指标综合评判模型
- 实测是最准确的性能评估方式**，注意测试环境与实际部署环境的差异，尽可能在实际部署的环境下测试性能
- 在设计模型初期就将推理性能作为设计指标，测试模型性能和迭代模型结构协同开展
- 深度学习推理模型需要从模型设计、推理框架、硬件选择**全流程优化**
- 遇到模型推理性能问题时，与HPC同学保持紧密沟通，具体问题具体分析

4.2 设计高效的深度学习网络结构

- 根据硬件平台的特性，调整网络的计算访存比（多平台部署可考虑设计多种模型）
 - 对于算力较弱的平台，受限于硬件计算能力，可以设计低计算量的网络来加快推理
 - 对于算力强的平台，一味降低计算量不一定有收益，需要关注访存量
- 网络子结构尽量选用经典结构，以最大可能匹配图优化规则

- 算子尽量使用常用参数，软件会对其做特殊优化
 - Conv: F3S1、F1S1、F3S2、F1S2、DwConv F3S1、DwConv F3S2 等等
- 设计网络时，channel 数尽量选择 4/8/16 的倍数
 - 推理框架会使用特殊的数据排布，channel 数会向上取整

4.3 选用高效的推理框架

 <https://github.com/PaddlePaddle/Paddle-Lite>

参考

<http://agroup.baidu.com/paddle-infer/view/pdf/4290481>